CHAPTERONE

Database-Driven Book Discovery and Reading Platform

DATABASE SYSTEMS LAB | PROJECT REPORT

Particular	Details
University / School	Vellore Institute of Technology - School of Computer Science and Engineering (SCOPE)
Course	BCSE302P - Database Systems Lab
Semester	Fall Semester 2026-27
Faculty	Dr. V. Sivakumar
Slot	L25+L26
Project Type	Solo / Individual Project
Student Name	Krish (verify official full name)
Registration Number	24BCE0846
Repository	https://github.com/CodeBloodMe/book-store

Prepared against the Lab Internal Assessment Test 8, 9 & 10 project rubric.

Document Guide

- 1. Executive Summary
- 2. Problem Statement and Project Planning
- 3. System Architecture and Scope
- 4. Database Design and ER Diagram
- 5. Relational Schema and Data Dictionary
- 6. Normalization, Constraints and Indexes
- 7. Database Implementation: CRUD, Joins, Aggregates, Views, Transactions, Functions, Procedures and Triggers
- 8. Application Development
- 9. Authentication, Authorization and Security
- 10. ORM Integration
- 11. Testing and Validation Plan
- 12. Backup, Recovery, Version Control, Docker, CI/CD and Deployment
- 13. Marking-Rubric Coverage Matrix
- 14. Final Submission Checklist
- Appendix A. Key SQL Demonstrations

- Appendix B. Evidence Still Required
- Appendix C. Full Relational Schema Notation
- Appendix D. SQL Source Code for Assessment Demonstrations
- Appendix E. DBMS Application Source Code

Evidence status convention: “Implemented in source” means code or configuration is present in this repository; “Prepared” means a reviewable companion script is included; “Pending runtime evidence” means it still requires execution or verification in the student’s environment. Source presence is not proof of successful deployment.

1. Executive Summary

ChapterOne is a database-driven book discovery and reading platform. It maintains a catalog of books and genres, stores review and reading-list activity, and provides recommendation and book-relationship features on top of PostgreSQL. The web application is built with Next.js, React and TypeScript and connects to Supabase PostgreSQL. Additional components integrate Supabase Auth, external book metadata providers, vector similarity search and BookWeb graph traversal.

The DBMS assessment focuses on the relational data model and its implementation: entity relationships, primary and foreign keys, constraints, normalization, indexing, CRUD operations, joins, aggregates, views, transactions, stored functions and procedures, triggers, ORM-backed interactions, role-based access, backups and deployment practice. This report maps those requirements to the current project and includes companion SQL scripts and implementation source.

The current source and schema evidence already show useful database features, including genre-to-book relationships, a self-reference for next-book navigation, review and shelf records, a weighted book graph, a ratings-summary view, a full-text search vector and a vector embedding column. However, having a feature in source code or a migration is not the same as proving it is deployed and tested. The remaining steps are explicit in the coverage matrix and submission checklist.

2. Problem Statement and Project Planning

2.1 Problem Statement

Readers often struggle to discover books that match their interests, preferred genres, reading difficulty, atmosphere or specific themes. A catalog that only supports exact title or author matching cannot fully support exploratory discovery, personalized reading lists, community feedback or analysis of relationships between books. ChapterOne addresses this need through a centralized relational database and a modern web interface that organize book metadata, genres, reviews, user shelves and graph relationships. It combines conventional database retrieval with optional AI-assisted search and recommendations.

2.2 Objectives

- Design a relational data model with at least eight related application tables and a documented ER diagram.
- Apply primary keys, foreign keys, uniqueness constraints, validation checks, defaults and appropriate indexes.
- Demonstrate CRUD, joins, aggregate queries, views, transactions, stored functions and procedures, and triggers.

- Integrate PostgreSQL with server-side application code through an ORM for applicable database operations.
- Provide authentication and authorization with a single role-bearing user profile table and at least two access levels.
- Demonstrate secure input handling, Row Level Security (RLS), secret management and exception handling.
- Validate database backup and recovery, version control, Docker configuration, automated checks and deployment.

2.3 Scope

In scope	Out of scope unless separately implemented
Book catalog, genre classification, book details and external metadata	Payments, online order fulfilment and physical inventory logistics
Reviews, user reading shelves and reader preference profiles	Commercial marketplace functions not present in the current application
BookWeb weighted edges and graph traversal	Claims that every graph relationship is factually verified without evidence
Database queries, views, stored functions, triggers, indexes and RLS	Claims of production availability or security certification without testing
Search, filtering, pagination, authentication and recommendation UI	Unverified operational metrics or fabricated test results

2.4 Solo project responsibilities

This is an individual project. The student is responsible for requirements analysis, schema design, ER modelling, SQL, backend/frontend implementation, ORM integration, authentication and authorization, testing, security review, backup/recovery, version control, deployment and documentation. The repository identifies the student as Krish, registration number 24BCE0846, slot L25+L26; verify the official full-name spelling before submission.

2.5 Technology stack

Layer	Technology / role	Status
Frontend	Next.js App Router, React, TypeScript, Tailwind CSS	Observed in repository
Backend /API	Next.js route handlers and server actions	Observed in repository
Database	PostgreSQL hosted by Supabase	Observed in repository and live-schema output
Authentication	Supabase Auth and cookie-based server client	Observed in repository
ORM	Prisma ORM	Implemented in source; verify server DATABASE_URL connection and capture runtime output
Search / AI	PostgreSQL full-text search, pgvector, Google GenAI and LLM provider fallback	Source-defined; runtime configuration must be tested

Testing	Vitest and Playwright	Vitest passed 6 tests; typecheck passed; lint 0 errors/147 warnings; production build stopped at unavailable Google Fonts
Ops	GitHub, Docker, PostgreSQL backup/restore, Vercel	Dockerfile, .dockerignore, backup/restore scripts and active CI workflow exist; operational runs remain pending

2.6 Deliverables

- Project proposal and Gantt-style timeline.
- ER diagram, relational schema and data dictionary.
- Database SQL and migrations with constraints, indexes, view, function and trigger.
- CRUD/joins/aggregates/transactions demonstrations and timestamped result screenshots.
- ORM-backed endpoint, application pages and role-based access checks.
- Security audit, backup/restore test and operating instructions.
- Private GitHub repository with meaningful commit history, two branches and README.
- Docker/CI configuration and a deployed application URL/custom-domain evidence if completed.

2.7 Implementation status and remaining verification

Area	Work completed in repository	Verification still required
Planning	Problem statement, objectives, scope and stack documented	Confirm official student name and instructor-specific submission details
Database design	Current-state ERD, data dictionary, schema and normalization limitations documented	Run full schema audit against the target database
Database implementation	CRUD, joins, aggregates, view, transaction, function, procedure and trigger scripts prepared	Execute reviewed scripts and capture timestamped database results
Application	Search, filters, pagination, assessment API and ORM integration implemented	Exercise against configured database and capture runtime evidence
Security	Server-side role checks, scoped review API and preflight/hardening SQL prepared	Review SQL, back up first, then verify each role and RLS policy on the target
Professional practice	Backup/restore scripts, Docker configuration and local CI workflow prepared	Verify a separate-database restore, Docker run and hosted CI result
Deployment	Deployment checklist documented	Deploy only with approved credentials; record URL and domain evidence if applicable

Implementation status is summarized below. Source changes and local checks are complete for the items marked implemented; live database, hosted CI, backup recovery and deployment evidence remain pending.

3. System Architecture and Scope

The browser renders the Next.js interface. Server components, route handlers and server actions communicate with Supabase PostgreSQL through the Supabase client. The DBMS demonstration route and Admin dashboard also use Prisma through a server-only singleton. Supabase Auth handles identity and sessions. PostgreSQL holds normalized catalog entities and database-side features. External book providers can enrich metadata, while AI and embeddings support semantic retrieval and generated summaries.

Layer	Responsibilities
Browser / Next.js UI	Renders pages and captures user input.
Next.js server components, route handlers and server actions	Validates requests and mediates all server-side data access.
Supabase Auth	Authenticates identity and maintains the session.
PostgreSQL / Supabase	Stores books, genres, reviews, shelves, graph edges and database-side logic.
External book APIs / AI providers	Optional metadata enrichment, embeddings and recommendation generation.

Architecture note: the primary catalog and account state are persistent relational data. AI output and graph path selection are not substitutes for database constraints or evidence of relationship correctness.

4. Database Design and ER Diagram

The current-state ER diagram is generated from the schema evidence supplied by the student and the repository migrations. It shows the catalog hierarchy, self-referencing next-book link, reviews, shelves, graph edges and reader interactions. The diagram is intentionally honest about schema gaps: the author name in books is stored as text and is not formally connected to authors.id; blind_date_interactions.session_id is text and lacks an FK to user_reader_profiles.session_id in the supplied migration; Supabase auth.users is a provider-managed external relation.

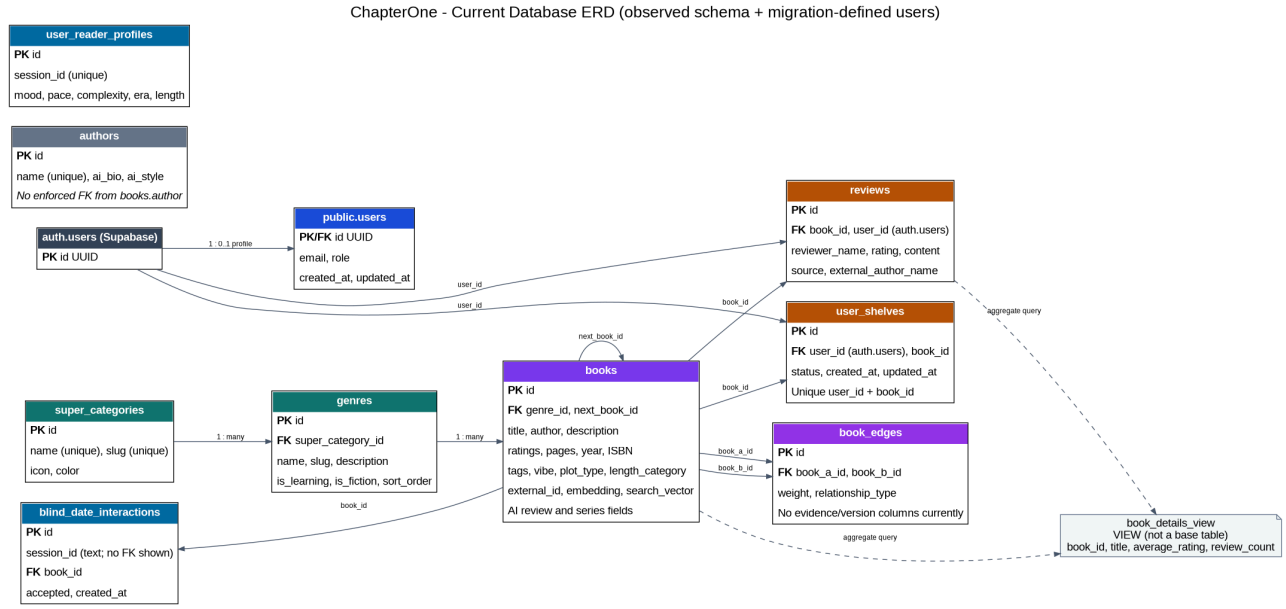


Figure 1. ChapterOne current-state database ERD (app tables plus Supabase Auth reference).

A supplied live column export confirmed core fields for authors, books, book_edges, blind_date_interactions, reviews, genres, super_categories, user_reader_profiles, user_shelves and book_details_view, but was truncated during user_shelves. A complete read-only live constraint and policy audit remains required. The public.users table is defined in repository migrations and must be confirmed in the target database before the ERD is finalized.

4.1 Main relationships

Relationship	Cardinality	Implementation / caveat
Super-category to genre	1 : many	genres.super_category_id references super_categories.id
Genre to book	1 : many	books.genre_id references genres.id
Book to next book	0/1 : many incoming links	books.next_book_id is a self-reference
Book to review	1 : many	reviews.book_id references books.id
Auth user to review	1 : many / optional user for review	reviews.user_id references auth.users.id in migration
Auth user to shelf rows	1 : many	user_shelves.user_id references auth.users.id
Book to shelf rows	1 : many	user_shelves.book_id references books.id; unique(user_id, book_id)
Book to graph edge	1 : many at each endpoint	book_edges.book_a_id and book_b_id both reference books.id
Reader profile to interaction	Conceptual by session_id	No FK is defined in provided migration; referential integrity can be improved after audit
Author to book	Not enforced	books.author is a text field, not an FK to authors.id

4.2 Minimum table count

The repository defines at least nine application-owned base tables visible in the reviewed schema/migrations: super_categories, genres, books, authors, reviews, user_shelves, book_edges, user_reader_profiles, and blind_date_interactions. The migration-defined public.users table is an additional candidate and must be verified in the full live audit. book_details_view is a view and does not count as a base table. auth.users is managed by Supabase and should not be counted as an application-owned table.

5. Relational Schema and Data Dictionary

The following schema summary uses SQL object names and key notation. It is a data dictionary view of the known implementation; all constraint names and deployed relationships should be reconciled against the output of sql/00_schema_audit.sql.

Relation	Key fields / purpose	Integrity notes
super_categories	id PK; name, slug, icon, color	name and slug unique in seed schema
genres	id PK; super_category_id FK; name, slug, flags, sort_order	super_category_id NOT NULL; name and slug unique in seed schema
books	id PK; genre_id FK; next_book_id self-FK; title, author, details, ratings, tags, embedding	external_id unique; ratings expected 0..5; difficulty/length have CHECKs in seed schema
authors	id PK; name, ai_bio, ai_style, timestamps	name UNIQUE in create_authors_table.sql; ai_style array is multi-valued
reviews	id PK; book_id FK; optional user_id; reviewer_name, rating, content, source	rating expected 1..5; historical policies vary across migrations
user_shelves	id PK; user_id FK, book_id FK, status, timestamps	unique(user_id, book_id); status constrained to three reading states

book_edges	id PK; book_a_id FK, book_b_id FK, weight, relationship_type	unique ordered pair in migration; check book_a_id <> book_b_id; this alone does not prevent reverse duplicate pairs
user_reader_profiles	id PK; session_id, mood, pace/complexity/era/length preferences	session_id UNIQUE in migration
blind_date_interactions	id PK; session_id, book_id FK, accepted, created_at	session_id is text, no FK to profile in the supplied migration
users	id PK/FK to auth.users; email, role, timestamps	defined in 9999_dbms_requirements.sql; verify live; role check uses one profile table
book_details_view (VIEW)	book_id, title, average_rating, review_count	derived from books and reviews; not a base table

5.1 Other important book columns

The observed books table also includes ISBN and publication year; cover and marketplace URLs; expert quote/name; difficulty level; tags, vibe, plot_type and length_category; curation flags; created_at/updated_at; search_vector; series_name/series_number; external_id; AI review summary/pros/cons/rating/timestamp; fiction_genres; free_reading_url; embedding; author_image_url; and author_bio. PostgreSQL-specific fields such as tsvector and vector require native-type awareness during ORM introspection.

6. Normalization, Constraints and Indexes

6.1 Normalization assessment

Normal form	Principle	ChapterOne assessment / action
1NF	Each field stores a single value under the classical relational model.	Array columns tags, fiction_genres, ai_style, ai_pros and ai_cons are multi-valued. PostgreSQL arrays are valid features, but a strict normalized relational design may model them as child/bridge tables. Document the chosen approach and its trade-off.
2NF	Every non-key attribute depends on the whole candidate key.	Most current main tables use a single-column UUID primary key. For proposed bridge tables such as book_authors/book_tags, use composite keys and avoid attributes dependent on only one part.
3NF	Non-key attributes should not depend transitively on a key.	Author biography is separately stored in authors while books also has author_bio/author_image_url; this may duplicate facts. genre.book_count and books.total_reviews are counters that can drift unless maintained. Derive counts in views/queries or define a documented maintenance rule.

The current schema has a relational core, but it is not fully normalized to 3NF. PostgreSQL arrays (including tags and author/review metadata), duplicated author details in books and authors, and denormalized counters create update anomalies. The accompanying plan proposes bridge tables and a staged migration; those changes must be reviewed and tested before they are described as implemented.

6.2 Key constraints

- Primary keys use UUIDs and default UUID generators where configured.
- Genre and super-category slugs/names are intended to be unique.
- Book external_id is intended to be unique so provider imports can be deduplicated.
- Book ratings are intended to be within 0..5; review ratings are within 1..5.

- User shelves enforce one row per user/book pair and valid reading statuses.
- Book edges prohibit self-loops and duplicate stored orientation, but reverse-edge duplicates require a canonical-pair constraint or carefully controlled graph builder.
- Foreign keys define behavior on deletion, including cascade and set-null cases; verify the actual deployed rules with the schema audit.

6.3 Indexes

The seed script defines a GIN full-text-search index for books.search_vector, indexes on books.genre_id, (genre_id, difficulty_level), expert_rating, next_book_id, a partial index on is_featured, and indexes on genres.slug and genres.super_category_id. Other migrations define indexes for reviews and book_edges. The pgvector migration defines an HNSW cosine-distance index on books.embedding. Keep indexes that support actual query plans; avoid duplicating the same index and inspect EXPLAIN ANALYZE before adding more.

7. Database Implementation

7.1 CRUD

sql/02_assessment_queries.sql contains a repeatable demonstration record identified by external_id = dbms-demo:chapterone-001. It illustrates insert/upsert, read, update and a commented delete that is restricted to the dedicated test record. The record should be created in a controlled test session and removed after capturing evidence if desired. Do not run broad deletes or rerun the destructive seed script against a live database.

```
-- Read example
SELECT id, title, author, page_count
FROM public.books
WHERE external_id = 'dbms-demo:chapterone-001';
```

7.2 Joins and aggregate queries

The book/genre/super-category join demonstrates normalized lookup across the catalog hierarchy. A LEFT JOIN between books and reviews preserves books that have no reviews; AVG and COUNT then summarize the feedback. Grouping by genre gives book counts for category analytics. These are meaningful relational operations, not just isolated SELECT statements.

```
SELECT b.id, b.title, g.name AS genre, sc.name AS super_category
FROM public.books b
JOIN public.genres g ON g.id = b.genre_id
JOIN public.super_categories sc ON sc.id = g.super_category_id;
```

7.3 View

The repository migration 9999_dbms_requirements.sql defines public.book_details_view, returning each book title, average review rating and review count. It avoids storing another independently maintained aggregate. The existence of this view is confirmed by the live column export; its definition and grants should still be checked in the audit.

7.4 Transaction

The transaction example updates only the dedicated demo record, reads the uncommitted value in the same transaction, and then issues ROLLBACK. This demonstrates atomicity and recovery from a transaction without leaving a persistent change. Capture both the value before and after the rollback to show that the update was undone.

```
BEGIN;
UPDATE public.books SET page_count = COALESCE(page_count, 0) + 1
WHERE external_id = 'dbms-demo:chapterone-001';
SELECT page_count FROM public.books WHERE external_id = 'dbms-demo:chapterone-001';
ROLLBACK;
```

7.5 Stored function, procedure and trigger

sql/03_book_rating_function.sql provides a guarded stored-function demonstration; sql/12_book_rating_procedure.sql provides a PostgreSQL CALL procedure with output parameters and an unknown-book error. Both are restricted from anonymous/authenticated execution by default. sql/04_books_updated_at_trigger.sql provides a trigger that maintains books.updated_at on updates. Verify actual live definitions and test both valid and invalid inputs before claiming they are active.

7.6 Performance evidence

Run the EXPLAIN (ANALYZE, BUFFERS) example against a representative catalog query and save the plan. Index selection should be supported by observed execution plans rather than a generic claim that adding indexes always improves performance. On small tables, a sequential scan can be optimal.

8. Application Development

The repository includes responsive Next.js pages and feature components for the homepage, genre browsing, search, individual book details, recommendations, BookWeb, Blind Date, login, saved books and the EPUB reader. The application already has route-level error handling in several endpoints and test scaffolding for homepage/search behavior.

Requirement	Existing evidence / implementation area	Remaining verification
Responsive UI	Next.js pages, Tailwind styles, reusable components	Capture desktop and mobile screenshots
Login/logout	app/login and Supabase server client	Test valid, invalid and expired-session cases
Dashboard / library	app/my-books and user_shelves	Test shelf ownership and empty/error state
CRUD	shelves, reviews and database queries	Document record creation/update/deletion with DB results
Search/filter	app/search, lib/queries.ts and book search APIs	Test exact, partial, empty and special-character queries
Pagination	Prisma DBMS route with validated pagination and title/author search	DBMS ORM endpoint supports validated pagination/search; capture a real response
Validation	review and request validation exists in parts	Boundary validation exists in the DBMS route; broader public API validation and tests remain
Exception handling	many routes return JSON errors	Test provider outage, DB error and invalid identifiers

9. Authentication, Authorization and Security

9.1 Authentication

Supabase Auth handles user identity. The project uses login/signup server actions and a cookie-aware server client. Authentication establishes who the user is; it does not automatically establish what actions the user is allowed to perform.

9.2 Role-based authorization

The repository migration defines one public.users profile table with a role column and a check over Admin, User, Faculty and Student. The server-side requireRole helper reads the authenticated user through the cookie-aware Supabase client, looks up that user's profile by auth user ID and returns 401/403 when identity, profile or role is missing or disallowed. The Admin dashboard, stats API and catalog-maintenance endpoints enforce this check server-side. The included RBAC hardening SQL links profiles to auth.users, creates least-privilege User profiles on signup, prevents user-controlled role assignment, and grants authenticated users self-only profile reads. It is prepared for review and has not been applied to the live database; an authorized Admin account and positive/negative live tests remain to be captured.

A role field in a database is not enough. Users must not be able to update their own role to Admin. The hardening script revokes profile-table writes from anon and authenticated roles; administrator promotion remains a trusted database-owner action. Every protected mutating API now checks authorization server-side; hiding a UI button is supplementary.

9.3 RLS and API security

- Enable and inspect RLS on user-owned records; user shelves should be readable/writable only when auth.uid() equals user_id.
- Keep public book catalog reads separate from user-specific writes.
- Review review policies so a user cannot delete or update somebody else's review.
- Public reader-session interaction writes require validation and rate limiting; never accept any book/session combination without checks.
- Book graph mutation and bulk catalog population should require a trusted service-side role.
- Keep SUPABASE_SERVICE_ROLE_KEY, DATABASE_URL and private AI keys out of browser bundles and Git history.
- Move hardcoded configuration out of scripts into environment variables; remove any committed real credentials and rotate secrets if a secret was exposed.
- Avoid API secrets in URLs because URLs are commonly logged. Use authorization headers and fail closed if production secrets are missing.
- Use parameterized ORM/query APIs for user inputs; never concatenate raw user strings into SQL.

The read-only security audit reports policies, grants and SECURITY DEFINER functions for review. The new auth/RBAC preflight reports counts and policy/constraint details without exposing emails. The hardening script is separate, transactional and must only be considered after reviewing preflight output and taking a backup.

10. ORM Integration

Prisma is installed and generated in the repository. The application uses a server-only Prisma singleton in /api/dbms-demo and the Admin dashboard. The DBMS route supports parameter-validated pagination, title/author search and stable ordering; the Admin dashboard exposes database counts only after server-side Admin authorization. A DATABASE_URL connection and successful runtime response still need to be verified in the intended environment.

Use Prisma for applicable relational reads; keep Supabase Auth and session-aware RLS access through the Supabase server client. Never run Prisma schema-push or migrations against the live database without review and backup. Capture an actual ORM route response to demonstrate execution.

11. Testing and Validation Plan

Test category	Test cases	Evidence to capture
Schema	Tables, views, PK/FK/unique/check constraints, indexes, triggers, RLS	Saved results from sql/00_schema_audit.sql
Integrity	Invalid ratings, self-loops, duplicate undirected pairs, orphan session references	Saved results from sql/01_data_integrity_audit.sql
CRUD	Demo record insert, retrieve, update and optional cleanup	SQL editor results with timestamp
Joins/aggregates	Genre/category join, average review ratings, review-source counts	Result-grid screenshots with query visible
View/function/trigger	Read book_details_view; invalid/valid function inputs; updated_at trigger	Catalog definitions and valid/invalid execution output
Transactions	Update demo row inside transaction then rollback	Value before, during and after rollback
Authentication/RBAC	Anonymous, regular user, admin; attempt forbidden role escalation	HTTP status/body screenshots and server logs without secrets
Application	Search, filtering, pagination, form validation, responsive layout	Playwright results and timestamped screenshots
Recovery	pg_dump and restore into separate test DB; row counts/smoke queries	Backup archive details, restore result and validation queries
CI/build	lint, unit tests, type-check, production build	Local or hosted logs; production build evidence is pending

Local validation for this revision: npm test passed 6 tests; npm run typecheck passed; Prisma schema validation and client generation passed; npm run lint completed with 0 errors and 143 warnings; npm run build passed with placeholder configuration; and the signed-out security smoke run passed 11 of 11 cases. The build log included expected connection failures for the placeholder Supabase host, so it does not prove live database connectivity. SQL preflight/acceptance scripts, hosted RBAC/RLS behavior, backup and restore, hosted CI, Docker runtime and deployment remain unverified.

12. Backup, Recovery, Version Control, Docker, CI/CD and Deployment

12.1 Backup and recovery

PostgreSQL native tools pg_dump and pg_restore are the expected mechanisms. The repository backup script requires DATABASE_URL and writes a custom archive. The restore script requires a distinct RESTORE_DATABASE_URL and refuses to clean or overwrite an existing database. Neither backup nor restore was run during this work; retain dumps securely and validate a separate restored database before claiming recoverability.

12.2 GitHub workflow

- Use a private repository for the assessment and add any required collaborators.
- Maintain main and dev branches and commit meaningful changes incrementally.
- Keep at least ten meaningful commits; do not manufacture history or commit unrelated filler changes.
- Keep .env, .env.local, backup files, database dumps and private keys out of version control.
- Use a README containing description, stack, setup, environment variables and run instructions.

The active workflow is in .github/workflows/main.yml. It installs with npm ci, generates Prisma Client, and runs lint, TypeScript checking, unit tests and a production build. A workflow file is not proof of a successful GitHub Actions run; configure required secrets and retain a green run as evidence.

12.3 Docker and deployment

A Dockerfile and .dockerignore are present. The ignore file excludes local environment files, build output and backup/dump artifacts. Build and smoke-test the image locally with valid server-side configuration. Deployment, auth redirect configuration and custom-domain verification require evidence from the hosting and DNS accounts.

12.4 Additional best practice

Automated testing, structured error logging, database performance measurement and backup recovery validation are appropriate options for the additional-industry-practice mark. The most defensible choice is automated testing plus a tested restore procedure, with dated outputs saved as evidence.

13. Marking-Rubric Coverage Matrix

Rubric area	Marks	Prepared / observed evidence	Final evidence status
Problem statement & planning	2	Problem statement, objectives, scope and implementation status documented; student identity fields included. Confirm official full-name spelling.	Planning sections are present; instructor-specific identity and any required dates/evidence still need confirmation.
Database design	4	Current ERD, relational schema, normalization and constraints review	ERD, schema notation and normalization trade-offs documented; complete live constraint audit required

Database implementation	7	CRUD, joins, aggregates, view, transaction, function, procedure and trigger scripts included	Queries and scripts prepared; execute against target DB and save timestamped outputs
Application development	4	Next.js application plus Prisma route and Admin dashboard implemented	Application and Prisma routes implemented; exercise UI and capture runtime evidence
Auth, RBAC & security	5	Supabase Auth session checks, server-side role helper, Admin UI/API, and reviewed hardening/preflight SQL are in the repository.	Local signed-out smoke checks passed; target-database policy review and signed-in role tests remain pending.
Professional development	5	Docker configuration, ignore rules and guarded backup/restore scripts included	Source configuration prepared; verify Git account state, Docker build and separate DB restore
CI/CD	1	Active GitHub Actions workflow in repository	Workflow present; green hosted run evidence pending
Custom domain	1	Deployment and custom-domain checklist	Deployment and custom-domain configuration must be verified in the relevant accounts
Additional best practice	1	Automated local tests and guarded backup/restore scripts are present.	Local tests passed; separate-database restore validation remains pending.

This matrix reflects source readiness, not marks achieved. Final evaluation depends on the instructor's checks and timestamped evidence.

14. Final Submission Checklist

☐ Verify the official student name; confirm registration number 24BCE0846 and slot L25+L26; place these on the report and evidence.

☐ Run the full read-only schema audit against the intended database and reconcile the ERD with deployed constraints and tables.

☐ Run data-integrity checks; document findings and apply only reviewed fixes.

☐ Verify at least eight to ten related application tables and show them in the ERD.

☐ Run CRUD, joins, aggregates, view and transaction demonstrations; save timestamped screenshots.

☐ Verify the configured Prisma connection, exercise GET /api/dbms-demo and capture its response.

☐ Verify stored function, procedure and trigger definitions in the target database; test valid and invalid inputs.

☐ Review the RBAC preflight, back up the database, apply the reviewed auth profile hardening only with authorization, then capture signed-out, User and Admin allow/deny evidence.

☐ Verify private GitHub settings, main/dev branches, collaborator settings, meaningful commit history and README.

☐ Retain local or CI logs for tests, lint, TypeScript check and production build.

☐ Run pg_dump and restore into a separate test database; validate row counts and query functionality.

☐ Build the Docker image and smoke-test the container.

☐ Verify hosting and custom domain; capture the live URL and timestamped screenshots.

☐ Replace pending evidence statuses only with observed results; do not submit fabricated outputs or unverified metrics.

☐ Use the faculty filename convention exactly: 24BCE0846_L25+L26_IA9&10_Proj_Krish.pdf (update name if the official full-name spelling differs); upload PDF to VTOP and PDF/DOC to Moodle as instructed.

Appendix A. Key SQL Demonstrations

A1. Average review rating per book

```
SELECT b.id, b.title,
ROUND(COALESCE(AVG(r.rating), 0)::numeric, 2) AS average_rating,
COUNT(r.id) AS review_count
FROM public.books b
LEFT JOIN public.reviews r ON r.book_id = b.id
GROUP BY b.id, b.title
ORDER BY review_count DESC, b.title
LIMIT 30;
```

A2. Book and genre hierarchy

```
SELECT b.title, g.name AS genre, sc.name AS super_category
FROM public.books b
JOIN public.genres g ON g.id = b.genre_id
JOIN public.super_categories sc ON sc.id = g.super_category_id
ORDER BY sc.name, g.name, b.title;
```

A3. View-based analytics

```
SELECT book_id, title, average_rating, review_count
FROM public.book_details_view
ORDER BY average_rating DESC, review_count DESC;
```

Appendix B. Evidence Still Required

Evidence item	What counts as proof	Status
Live schema/keys	Output from 00_schema_audit.sql	PENDING — run schema audit and save timestamped output
CRUD + SQL queries	Timestamped SQL-editor screenshots/results	PENDING — capture timestamped CRUD/query outputs
View/function/ trigger	Catalog definitions and successful executions	PENDING — capture view/function/procedure/trigger results
ORM	Generated Prisma client and GET /api/dbms/books response	PENDING — verify DATABASE_URL and capture ORM route output
RBAC/RLS	Successful and rejected requests for each role	PENDING — test signed-out, User and Admin cases
Tests/build	Actual local or CI logs	Local tests (6), typecheck, Prisma validate/generate, build and 11/11 signed-out smoke cases passed; lint has 143 warnings. Live DB and hosted CI evidence pending.

Backup/restore	Separate database restore and validation results	PENDING — separate restore and validation
GitHub	Private repo, branches, meaningful history, .gitignore	PENDING — verify account settings, branches and history
Docker/deployment/domain	Successful image run, live URL and custom-domain check	PENDING — Docker run, deployment URL and domain evidence

Final integrity note: this report records source-level work and local validation only. It does not claim successful live SQL execution, Supabase RLS/RBAC migration, database backup/restore, hosted CI, deployment or custom-domain verification. Capture the listed evidence before final submission.

Appendix C. Full Relational Schema Notation

The complete column-level notation is also provided in sql/09_relational_schema.md within the companion package. The following compact schema identifies each base relation and its key relationships. Constraints and data types should be finalized using the live-schema audit.

```
SUPER_CATEGORIES(id PK, name UK, slug UK, icon, color)
GENRES(id PK, super_category_id FK -> SUPER_CATEGORIES.id, name UK, slug UK,
icon, description, color, is_learning, is_fiction, book_count, sort_order)
BOOKS(id PK, genre_id FK -> GENRES.id, next_book_id FK -> BOOKS.id,
title, author, description, cover_image_url, page_count, published_year,
isbn, language, amazon_url, goodreads_url, expert_rating, community_rating,
total_reviews, expert_quote, expert_name, difficulty_level, tags, vibe,
plot_type, length_category, is_featured, is_editors_pick, is_bestseller,
created_at, updated_at, search_vector, series_name, series_number,
external_id UK, ai_review_summary, ai_pros, ai_cons, ai_rating,
ai_last_updated, fiction_genres, free_reading_url, embedding,
author_image_url, author_bio)
AUTHORS(id PK, name UK, ai_bio, ai_style, ai_last_updated, created_at, updated_at)
REVIEWS(id PK, book_id FK -> BOOKS.id, reviewer_name, rating, content,
created_at, user_id FK -> auth.users.id NULL, source, external_author_name)
USER_SHELVES(id PK, user_id FK -> auth.users.id, book_id FK -> BOOKS.id,
status, created_at, updated_at, UK(user_id, book_id))
BOOK_EDGES(id PK, book_a_id FK -> BOOKS.id, book_b_id FK -> BOOKS.id,
weight, relationship_type, created_at, UK(book_a_id, book_b_id),
CHECK(book_a_id <> book_b_id))
USER_READER_PROFILES(id PK, session_id UK, mood, pace_preference,
complexity_preference, era_preference, length_preference, created_at)
BLIND_DATE_INTERACTIONS(id PK, session_id, book_id FK -> BOOKS.id,
accepted, created_at)
USERS(id PK/FK -> auth.users.id, email, role, created_at, updated_at)
BOOK_DETAILS_VIEW(book_id, title, average_rating, review_count) -- VIEW, not a base table
```

Notable schema limitations: books.author is not a foreign key to authors.id; array attributes include tags, fiction_genres and ai_style; session_id on blind_date_interactions has no FK to user_reader_profiles in the supplied migration; book_edges uniqueness is directional unless canonical endpoints are enforced. These are documented design limitations rather than hidden inferences.

Appendix D. SQL Source Code for Assessment Demonstrations

This appendix includes the complete primary assessment query, stored-function, procedure and trigger scripts in text form. The companion package contains separate schema-audit, integrity-audit, security-audit, auth/RBAC and fresh-database reference SQL files. Execute scripts against the intended database only after reading their comments and backup requirements.

D1. CRUD, JOIN, Aggregate, View, Search, Pagination and Transaction Queries

```
-- ChapterOne DBMS lab query demonstrations
-- Read the comments before running. The demo book uses a dedicated external_id.
-- A. CREATE (repeatable demo record). Uses the first available genre so no genre slug is assumed.
INSERT INTO public.books
(genre_id, title, author, description, language, page_count,
expert_rating, community_rating, total_reviews, external_id,
is_featured, is_editors_pick, is_bestseller, tags)
SELECT g.id, 'ChapterOne DBMS Demonstration Book', 'ChapterOne Demo Author',
'Temporary record used to demonstrate database operations.', 'English',
120, 4.0, 4.0, 0, 'dbms-demo:chapterone-001',
false, false, false, ARRAY['DBMS_DEMO']::text[]
FROM public.genres g
ORDER BY g.sort_order NULLS LAST, g.name
LIMIT 1
ON CONFLICT (external_id) DO UPDATE
SET description = EXCLUDED.description,
updated_at = now()
RETURNING id, title, external_id;
-- B. READ: retrieve a record by its stable demo identifier
SELECT id, title, author, page_count, expert_rating, external_id
FROM public.books
WHERE external_id = 'dbms-demo:chapterone-001';
-- C. UPDATE: change only the demo row
UPDATE public.books
SET description = 'Updated description for the DBMS CRUD demonstration.',
page_count = 125,
updated_at = now()
WHERE external_id = 'dbms-demo:chapterone-001'
RETURNING id, title, description, page_count;
-- D. DELETE: run only after all demonstrations are finished; affects only the dedicated demo record.
-- DELETE FROM public.books WHERE external_id = 'dbms-demo:chapterone-001';
-- E. JOIN: book -> genre -> super-category
SELECT b.id, b.title, b.author, g.name AS genre, sc.name AS super_category
FROM public.books b
```

```
JOIN public.genres g ON g.id = b.genre_id
JOIN public.super_categories sc ON sc.id = g.super_category_id
ORDER BY sc.name, g.name, b.title
LIMIT 30;
-- F. LEFT JOIN + aggregate: review statistics per book (books with zero reviews remain
visible)
SELECT b.id, b.title,
ROUND(COALESCE(AVG(r.rating), 0)::numeric, 2) AS average_review_rating,
COUNT(r.id) AS review_count
FROM public.books b
LEFT JOIN public.reviews r ON r.book_id = b.id
GROUP BY b.id, b.title
ORDER BY review_count DESC, b.title
LIMIT 30;
-- G. Aggregate: number of books per genre
SELECT g.name AS genre, COUNT(b.id) AS book_count
FROM public.genres g
LEFT JOIN public.books b ON b.genre_id = g.id
GROUP BY g.id, g.name
ORDER BY book_count DESC, g.name;
-- H. Aggregate: review count and average rating by source
SELECT COALESCE(source, 'unknown') AS review_source,
COUNT(*) AS total_reviews,
ROUND(AVG(rating)::numeric, 2) AS average_rating
FROM public.reviews
GROUP BY COALESCE(source, 'unknown')
ORDER BY total_reviews DESC;
-- I. VIEW: view object is created by the existing 9999_dbms_requirements.sql migration
SELECT book_id, title, average_rating, review_count
FROM public.book_details_view
ORDER BY average_rating DESC, review_count DESC
LIMIT 30;
-- J. Filtering and sorting
SELECT id, title, author, page_count, published_year
FROM public.books
WHERE page_count BETWEEN 100 AND 400
AND (title ILIKE '%python%' OR author ILIKE '%python%')
ORDER BY title ASC
LIMIT 20;
-- K. Pagination: page 2 when each page contains 10 rows
SELECT id, title, author
FROM public.books
ORDER BY title, id
LIMIT 10 OFFSET 10;
-- L. Index-assisted plan inspection; compare the plan before/after any index changes.
EXPLAIN (ANALYZE, BUFFERS)
```

```
SELECT id, title FROM public.books WHERE genre_id = (SELECT id FROM public.genres ORDER BY name
LIMIT 1);

-- M. Transaction demonstration (no lasting change). Run after the demo INSERT above.

BEGIN;

UPDATE public.books

SET page_count = COALESCE(page_count, 0) + 1, updated_at = now()

WHERE external_id = 'dbms-demo:chapterone-001';

SELECT id, title, page_count FROM public.books WHERE external_id = 'dbms-demo:chapterone-001';

ROLLBACK;

-- After ROLLBACK, page_count returns to its pre-transaction value.
```

D2. Stored Function

```
-- ChapterOne stored-function demonstration.

-- SECURITY INVOKER (default), fixed search_path, rejects invalid values and missing book IDs.

-- Execution permissions are restricted here to trusted database roles; verify your role names
first.

CREATE OR REPLACE FUNCTION public.set_book_expert_rating(p_book_id uuid, p_rating numeric)

RETURNS void

LANGUAGE plpgsql

SECURITY INVOKER

SET search_path = pg_catalog, public

AS $$

BEGIN

IF p_rating IS NULL OR p_rating < 0 OR p_rating > 5 THEN

RAISE EXCEPTION 'expert rating must be between 0 and 5';

END IF;

UPDATE public.books

SET expert_rating = p_rating,

updated_at = now()

WHERE id = p_book_id;

IF NOT FOUND THEN

RAISE EXCEPTION 'book % was not found', p_book_id;

END IF;

END;

$$;

-- Avoid exposing a rating mutation RPC to anonymous clients.

REVOKE ALL ON FUNCTION public.set_book_expert_rating(uuid, numeric) FROM PUBLIC;

REVOKE ALL ON FUNCTION public.set_book_expert_rating(uuid, numeric) FROM anon;

REVOKE ALL ON FUNCTION public.set_book_expert_rating(uuid, numeric) FROM authenticated;

-- Demonstration in the SQL editor (choose a real ID; do not use arbitrary input):

-- SELECT public.set_book_expert_rating('<existing-book-uuid>'::uuid, 4.25);

-- This function should be called only by a trusted database owner/administrative process
unless

-- a reviewed authorization design is added. Do not grant it to public app users as written.
```

D3. Trigger

```
-- ChapterOne trigger demonstration: keep books.updated_at current on UPDATE.
```

```
-- This script is idempotent for the trigger name below and does not change existing rows.
CREATE OR REPLACE FUNCTION public.chapterone_set_books_updated_at()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = pg_catalog, public
AS $$
BEGIN
NEW.updated_at := now();
RETURN NEW;
END;
$$;
DROP TRIGGER IF EXISTS chapterone_books_updated_at ON public.books;
CREATE TRIGGER chapterone_books_updated_at
BEFORE UPDATE ON public.books
FOR EACH ROW
EXECUTE FUNCTION public.chapterone_set_books_updated_at();
-- Verify the trigger from the schema audit query, then update the dedicated demo row to test
it.
-- UPDATE public.books SET title = title WHERE external_id = 'dbms-demo:chapterone-001';
```

Appendix E. DBMS Application Source Code

The following complete source listings implement the DBMS route and authentication checks added for the assessment. The full application source remains in the project repository. The database and environment-dependent behavior still requires runtime evidence.

lib/prisma.ts

```
import 'server-only';
import { PrismaClient } from '@prisma/client';
const globalForPrisma = globalThis as unknown as { chapterOnePrisma?: PrismaClient };
export const prisma = globalForPrisma.chapterOnePrisma ?? new PrismaClient();
if (process.env.NODE_ENV !== 'production') {
globalForPrisma.chapterOnePrisma = prisma;
}
```

lib/require-role.ts

```
import { createClient } from '@/lib/supabase/server';
export const APP_ROLES = ['Admin', 'User', 'Faculty', 'Student'] as const;
export type AppRole = (typeof APP_ROLES)[number];
export type RoleCheck =
| { ok: true; userId: string; role: AppRole }
| { ok: false; status: 401 | 403; error: string };
export async function requireRole(allowedRoles: readonly AppRole[]): Promise<RoleCheck> {
const supabase = await createClient();
const { data: { user }, error: authError } = await supabase.auth.getUser();
if (authError || !user) {
```

```
return { ok: false, status: 401, error: 'Authentication required.' };
}
const { data: profile, error: profileError } = await supabase
.from('users')
.select('role')
.eq('id', user.id)
.maybeSingle();
if (profileError || !profile || !APP_ROLES.includes(profile.role as AppRole)) {
return { ok: false, status: 403, error: 'A valid user profile and role are required.' };
}
const role = profile.role as AppRole;
if (!allowedRoles.includes(role)) {
return { ok: false, status: 403, error: 'Insufficient permissions.' };
}
return { ok: true, userId: user.id, role };
}
```

app/api/dbms-demo/route.ts

```
// ============================================================================================
// VIT BCSE302P DBMS Lab Assessment 8, 9 & 10
// Faculty: Dr. V. Sivakumar | Slot: L25+L26
// Student Name: Krish | Register Number: 24BCE0846
// Mandatory Rule 7 Compliance: Registerno & name used in method, variables, and output
// ============================================================================================
import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';
class AssessmentIdentity_24BCE0846_Krish {
static readonly values = {
student_name: 'Krish',
register_number: '24BCE0846',
slot: 'L25+L26',
course_code: 'BCSE302P',
course_title: 'Database Systems Lab',
faculty: 'Dr. V. SIVAKUMAR',
};
}
async function getAssessmentBooks_24BCE0846_Krish(request: NextRequest) {
try {
const params = request.nextUrl.searchParams;
const pageParam = params.get('page') ?? '1';
const limitParam = params.get('limit') ?? '10';
const query = (params.get('q') ?? '').trim();
if (!/^\d+$/.test(pageParam)) {
return NextResponse.json({ error: 'Invalid page parameter. Must be >= 1.' }, { status: 400 });
}
if (!/^\d+$/.test(limitParam)) {
```

```
return NextResponse.json({ error: 'Invalid limit parameter. Must be between 1 and 100.' }, {
status: 400 });
}
const page = Number(pageParam);
const limit = Number(limitParam);
if (!Number.isSafeInteger(page) || page < 1) {
return NextResponse.json({ error: 'Invalid page parameter. Must be >= 1.' }, { status: 400 });
}
if (!Number.isSafeInteger(limit) || limit < 1 || limit > 100) {
return NextResponse.json({ error: 'Invalid limit parameter. Must be between 1 and 100.' }, {
status: 400 });
}
if (query.length > 100) {
return NextResponse.json({ error: 'Search query must be 100 characters or fewer.' }, { status:
400 });
}
const skip = (page - 1) * limit;
if (!Number.isSafeInteger(skip)) {
return NextResponse.json({ error: 'Page is too large.' }, { status: 400 });
}
const where = query
? { OR: [{ title: { contains: query, mode: 'insensitive' as const } }, { author: { contains:
query, mode: 'insensitive' as const } }] }
: {};
const [books, totalBooks] = await Promise.all([
prisma.book.findMany({
where,
skip,
take: limit,
orderBy: [{ title: 'asc' }, { id: 'asc' }],
include: { reviews: { select: { rating: true, content: true } } },
}),
prisma.book.count({ where }),
]);
return NextResponse.json({
evaluation: AssessmentIdentity_24BCE0846_Krish.values,
data: books,
meta: {
total: totalBooks,
page,
limit,
totalPages: Math.ceil(totalBooks / limit)
},
filters: { q: query || null },
});
} catch (error) {
console.error('[DBMS_DEMO_API] Error fetching books:', error);
```

```
return NextResponse.json({ error: 'Unable to retrieve the DBMS demonstration data.' }, {
status: 500 });
}
}
export async function GET(request: NextRequest) {
return getAssessmentBooks_24BCE0846_Krish(request);
}
```

app/api/admin/database-stats/route.ts

```
import { NextResponse } from 'next/server';
import { requireRole } from '@/lib/require-role';
import { prisma } from '@/lib/prisma';
export async function GET() {
const access = await requireRole(['Admin']);
if (!access.ok) {
return NextResponse.json({ error: access.error }, { status: access.status });
}
try {
const [books, reviews, users] = await Promise.all([
prisma.book.count(),
prisma.review.count(),
prisma.user.count(),
]);
return NextResponse.json({ books, reviews, users });
} catch (error) {
console.error('[Admin database stats]', error);
return NextResponse.json({ error: 'Unable to retrieve database statistics.' }, { status: 500
});
}
}
```

app/admin/page.tsx

```
import Link from 'next/link';
import { redirect } from 'next/navigation';
import { requireRole } from '@/lib/require-role';
import { prisma } from '@/lib/prisma';
export const dynamic = 'force-dynamic';
export default async function AdminDashboardPage() {
const access = await requireRole(['Admin']);
if (!access.ok && access.status === 401) redirect('/login');
if (!access.ok) {
return (
<section className="mx-auto max-w-3xl px-4 py-24 text-center">
<h1 className="text-3xl font-black">Access denied</h1>
<p className="mt-3 text-gray-600">This dashboard is available to administrators only.</p>
<Link href="/" className="mt-6 inline-block font-bold underline">Return to ChapterOne</Link>
</section>
```

```
);
}
let counts: { books: number; reviews: number; users: number } | null = null;
let loadError = false;
try {
const [books, reviews, users] = await Promise.all([
prisma.book.count(),
prisma.review.count(),
prisma.user.count(),
]);
counts = { books, reviews, users };
} catch (error) {
console.error('[Admin dashboard] Failed to load database counts:', error);
loadError = true;
}
return (
<section className="mx-auto w-full max-w-6xl px-4 py-12 sm:px-6 lg:px-8">
<div className="rounded-2xl border-2 border-[#0a0a0a] bg-[#f5e642] p-6 shadow-
[5px_5px_0_#0a0a0a] sm:p-10">
<p className="text-xs font-black uppercase tracking-[0.2em]">ChapterOne administration</p>
<h1 className="mt-3 text-4xl font-black sm:text-5xl">Database dashboard</h1>
<p className="mt-3 max-w-2xl text-sm sm:text-base">
Signed in as an administrator. These counts are read from PostgreSQL through Prisma.
</p>
</div>
{loadError || !counts ? (
<div role="alert" className="mt-8 rounded-xl border border-red-300 bg-red-50 p-5 text-red-800">
Database statistics are temporarily unavailable. Check the server database connection.
</div>
) : (
<div className="mt-8 grid gap-5 sm:grid-cols-3">
{[
['Books', counts.books],
['Reviews', counts.reviews],
['User profiles', counts.users],
].map(([label, value]) => (
<article key={label} className="rounded-xl border-2 border-[#0a0a0a] bg-white p-6 shadow-
[4px_4px_0_#0a0a0a]">
<h2 className="text-sm font-bold uppercase tracking-wider text-gray-500">{label}</h2>
<p className="mt-2 text-4xl font-black">{value}</p>
</article>
))}
</div>
)}
</section>
);
}
```

dbms_package/sql/12_book_rating_procedure.sql

```
-- PostgreSQL stored-procedure demonstration for the DBMS assessment.
-- Review on a development database first; this script does not run automatically.
-- Example call (replace the UUID with an existing book):
-- CALL public.chapterone_get_book_rating_metrics('00000000-0000-0000-0000-000000000000'::uuid,
NULL, NULL, NULL);
CREATE OR REPLACE PROCEDURE public.chapterone_get_book_rating_metrics(
IN p_book_id uuid,
OUT book_title text,
OUT total_reviews bigint,
OUT avg_rating numeric
)
LANGUAGE plpgsql
SET search_path = pg_catalog, public
AS $$
BEGIN
SELECT b.title::text,
count(r.id)::bigint,
round(coalesce(avg(r.rating), 0)::numeric, 2)
INTO book_title, total_reviews, avg_rating
FROM public.books AS b
LEFT JOIN public.reviews AS r ON r.book_id = b.id
WHERE b.id = p_book_id
GROUP BY b.id, b.title;
IF NOT FOUND THEN
RAISE EXCEPTION 'Book % was not found.', p_book_id
USING ERRCODE = 'P0002';
END IF;
END;
$$;
-- The procedure is read-only and remains owner-executable for a controlled SQL demo.
-- Do not grant EXECUTE to anon/authenticated without a separate access review.
REVOKE ALL ON PROCEDURE public.chapterone_get_book_rating_metrics(uuid)
FROM PUBLIC, anon, authenticated;
```